

LEARNING TO OPTIMIZE: TRAINING DEEP NEURAL NETWORKS FOR WIRELESS RESOURCE MANAGEMENT

摘要: DNN 逼近 WMMSE, 提高计算速度, 实现实时功率分配

关键词: DNN; WMMSE; 实时功率分配;

1 待优化问题

考虑一个包含 K 个单天线收发对的干扰信道 (IC), 用户 k 的 SINR 为:

$$\text{SINR}_k \triangleq \frac{|h_{kk}|^2 p_k}{\sum_{j \neq k} |h_{kj}|^2 p_j + \sigma_k^2}$$

优化目标是最大化加权和速率 (WSR):

$$\begin{aligned} \max_{p_1, \dots, p_K} \quad & \sum_{k=1}^K \alpha_k \log \left(1 + \frac{|h_{kk}|^2 p_k}{\sum_{j \neq k} |h_{kj}|^2 p_j + \sigma_k^2} \right) \\ \text{s.t.} \quad & 0 \leq p_k \leq P_{\max}, \quad \forall k = 1, 2, \dots, K \end{aligned}$$

这个问题是非凸优化问题 (NP-hard)。

2 WMSE 算法

核心是引入辅助变量, 将原始问题转化为关于三个变量 (接收机权重、发射功率和均衡系数) 的优化问题, 且转化后的问题在每一个变量上是凸的 (当固定其他两个变量时)

2.1 引入辅助变量 v_k 、 u_k

对用户 k 接收的信号 $\hat{s}_k$, 定义其最小均方误差 MMSE 的表达式:

$$\begin{aligned} e_k &\triangleq \mathbb{E}_{s, z} (\hat{s}_k - s_k)^2 \\ &= (u_k |h_{kk}| v_k - 1)^2 + \sum_{j \neq k} (u_k |h_{kj}| v_j)^2 + \sigma_k^2 u_k^2 \end{aligned}$$

这里注意一下 s_k 应该是零均值和方差为 1, 且彼此之间相互正交的。通过经典结论 MMSE-SINR 等式, 可知

$$\text{MMSE}_k = \frac{1}{1 + \text{SINR}_k} \Leftrightarrow \text{SINR}_k = \frac{1}{\text{MMSE}_k} - 1$$

SINR 越高, MMSE 越小? 或许是这个物理意义? 将等式代入速率公式可得:

$$\log(1 + \text{SINR}_k) = \log\left(\frac{1}{\text{MMSE}_k}\right) = -\log(\text{MMSE}_k)$$

因此原始问题转化为:

$$\max_{\{p_k\}} \sum_{k=1}^K -\alpha_k \log(\text{MMSE}_k) \Leftrightarrow \min_{\{p_k\}} \sum_{k=1}^K \alpha_k \log(\text{MMSE}_k)$$

2.2 引入辅助变量 w_k

通过引入辅助变量 $w_k > 0$, 我们可以建立以下关系式

$$-\log(\text{MMSE}_k) = \max_{w_k > 0} (\log w_k - w_k \cdot \text{MMSE}_k + 1)$$

证明这个等式: 令:

$$f(w_k) = \log w_k - w_k \cdot \text{MMSE}_k + 1$$

对 w_k 求导并令导数为 0:

$$\frac{\partial f}{\partial w_k} = \frac{1}{w_k} - \text{MMSE}_k = 0 \Rightarrow w_k^* = \frac{1}{\text{MMSE}_k}$$

代入得:

$$f(w_k^*) = \log\left(\frac{1}{\text{MMSE}_k}\right) - \frac{1}{\text{MMSE}_k} \cdot \text{MMSE}_k + 1 = -\log(\text{MMSE}_k)$$

因此:

$$\max_{w_k > 0} f(w_k) = -\log(\text{MMSE}_k) = R_k$$

因此这个问题被转化为了:

$$\begin{aligned} \min_{\{w_k, u_k, v_k\}} \quad & \sum_{k=1}^K \alpha_k (w_k e_k - \log(w_k)) \\ \text{s.t.} \quad & 0 \leq v_k \leq \sqrt{P_k}, k = 1, 2, \dots, K. \end{aligned}$$

2.3 算法求解

引入三个辅助变量后, 可以通过交替优化 v_k 、 u_k 、 w_k 来求解这个问题, 问题被拆分成三个凸优化问题, 这里采用块坐标下降的方法来求解每个子问题。算法的伪代码如下

Algorithm 1: WMMSE 功率控制算法

- 1: **Initialize** v_k^0 满足 $0 \leq v_k^0 \leq \sqrt{P_k}, \forall k$;
 - 2: **compute** $u_k^0 = \frac{|h_{kk}|v_k^0}{\sum_{j=1}^K |h_{kj}|(v_j^0)^2 + \sigma_k^2}, \forall k$;
 - 3: **compute** $w_k^0 = \frac{1}{1 - u_k^0|h_{kk}|v_k^0}, \forall k$;
 - 4: **set** $t = 0$
 - 5: **repeat**
 - 6: $t = t + 1$ {iterations}
 - 7: **Update** v_k :

$$v_k^t = \left[\frac{\alpha_k w_k^{t-1} u_k^{t-1} |h_{kk}|}{\sum_{j=1}^K \alpha_j w_j^{t-1} (u_j^{t-1})^2 |h_{jk}|^2} \right]_0^{\sqrt{P_{\max}}}, \forall k$$
;
 - 8: **Update** u_k : $u_k^t = \frac{|h_{kk}|v_k^t}{\sum_{j=1}^K |h_{kj}|^2 (v_j^t)^2 + \sigma_k^2}, \forall k$;
 - 9: **Update** w_k : $w_k^t = \frac{1}{1 - u_k^t |h_{kk}|v_k^t}, \forall k$;
 - 10: **until** $\left| \sum_{j=1}^K \log(w_j^t) - \sum_{j=1}^K \log(w_j^{t-1}) \right| \leq \epsilon$
 - 11: **output** $p_k = (v_k^t)^2, \forall k$;
-

3 DNN 逼近 WMMSE 算法

3.1 为什么可以用 DNN 逼近 WMMSE 算法

3.1.1 迭代算法的通用近似定理

X 和 Y_0 是紧致集 (有界且闭的集合), 且每次迭代映射 f_t 都是连续函数。则对任意给定误差精度 $\epsilon > 0$, 存在一个足够大的正整数 N (隐藏神经元数量) 使得单隐藏层网络 $NET_N(y_0, x)$ 能以小于 ϵ 的误差一致地逼近复合函数 $F_T(y_0, x)$ 。

3.1.2 对于 WMMSE 算法

对于 WMMSE 算法的每一次迭代 f_t , 更新 u_k :

$$u_k = \frac{|h_{kk}|v_k}{\sum_{j=1}^K |h_{kj}|^2 v_j^2 + \sigma_k^2}$$

分母在实际应用场景下不为 0, 因此整个分式是 $|h_{kj}|$ 和 v_j 的连续函数。 u_k 的更新是连续映射。同理, v_k 和 w_k 的映射也是连续映射。对于投影操作 $\min$ 和 $\max$ 函数都是连续的, 也是一个连续映射。

因此每次迭代 f_t 都是一个连续映射。且信道参数 X 和初始功率 Y_0 在实际问题中是紧致的 (有界的)。这就从理

论上证明了: 存在一个单隐藏层神经网络, 可以任意精度逼近迭代有限次数的 WMMSE 算法从信道参数 X 和固定初始化 y_0 到最终功率分配 y_t 的映射关系 F_T 。

3.2 DNN 系统实现

3.2.1 网络结构

全连接前馈神经网络

- 输入层: 接收信道系数 $|h_{kj}|$
- 隐藏层: 使用三个隐藏层, 神经元数分别为 200、80、80, 激活函数选择 ReLU 函数。
- 输出层: 输出功率分配 p_k , 约束激活函数

$$y = \min(\max(x, 0), p_{\max})$$

3.2.2 数据生成 (Data Generation)

训练数据集通过以下流程生成:

1. 根据指定的分布生成信道实现 $\{h_{kj}^{(i)}\}$:
 - 高斯干扰信道 (Gaussian IC): $|h_{kj}| \sim \mathcal{N}(0, 1)$ (瑞利衰落)
 - 实际干扰多址接入信道 (Practical IMAC): $|h_{nk}| = (200/d_{nk})^3 \cdot L$, 其中 $L \sim$ 对数正态分布(0, 64) (d_{nk} 是用户 k 到基站 n 的距离)
2. 对于每个参数组合 $(P_{\max}, \{\sigma_k\}, \{|h_{kj}^{(i)}|\})$:
 - 运行 **WMMSE 算法** 获取优化后的功率向量 $\{p_k^{(i)}\}$
 - 初始化: $v_k^0 = \sqrt{P_{\max}} \quad \forall k$ (所有发射机初始功率设为最大功率)
 - 终止条件: $|\text{目标函数值}_{\text{新}} - \text{目标函数值}_{\text{旧}}| < 10^{-3}$ (目标函数变化小于阈值)
3. 构成训练样本: $(\{|h_{kj}^{(i)}|\}, \{p_k^{(i)}\})$ (信道系数作为输入, WMMSE 计算的功率作为标签)
4. 重复上述过程生成完整数据集:

总样本数: 1,000,000

训练集 $\mathcal{T}$: 99% (990,000 个样本)

验证集 $\mathcal{V}$: 1% (10,000 个样本)

3.2.3 训练阶段 (Training Stage)

使用训练集 $(\{|h_{kj}^{(i)}|\}, \{p_k^{(i)}\})_{i \in \mathcal{T}}$ 优化深度神经网络的权重参数:

- **损失函数 (Loss Function):** 均方误差 (Mean Squared Error - MSE)

$$\mathcal{L} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \sum_{k=1}^K (p_{k, \text{预测}}^{(i)} - p_k^{(i)})^2$$

(目标: 最小化 DNN 预测功率 $p_{k, \text{预测}}$ 与 WMMSE 计算功率标签 p_k 之间的均方差)

- **优化算法 (Optimization Algorithm):** RMSprop (均方根传播)

衰减率 (Decay rate) = 0.9

学习率 (Learning rate) = 0.001

批大小 (Batch size) = 1000

(RMSprop 是一种高效的 mini-batch 随机梯度下降变种, 根据梯度历史幅度自适应调整学习率)

- **训练技巧 (Training Techniques):**

1. 权重初始化 (Weight Initialization):

- 使用截断正态分布 (Truncated normal distribution) 初始化权重
- 方差缩放 (Variance scaling): $w \leftarrow w / \sqrt{\text{输入连接数 (fan_in)}}$ (归一化神经元输出方差, 加速训练)

2. 学习率调度 (Learning Rate Scheduling):

- 当验证集误差不再下降时, 逐步降低学习率 (主要应用于 IMAC 场景)

3. 批归一化 (Batch Normalization):

- 应用于大型 IMAC 场景 ($N = 7, K = 28$)
- 通过减少内部协变量偏移 (Internal covariate shift) 来加速深度网络训练

3.2.4 测试阶段 (Testing Stage)

在未见过的测试数据上评估训练好的 DNN 模型性能:

1. 按照与训练数据相同的分布生成测试信道 $\{h_{jk}^{(\text{测试})}\}$
2. 进行推理 (Inference):

$$\{p_{k, \text{DNN}}\} = \text{DNN}(\{|h_{kj}^{(\text{测试})}|\})$$

(将测试信道输入训练好的 DNN, 得到预测的功率分配 $\{p_{k, \text{DNN}}\}$)

3. 评估关键指标:

- **Sum-Rate Performance:**

$$R_{\text{DNN}} = \sum_{k=1}^K \alpha_k \log_2 (1 + \text{信干噪比}_k^{(\text{DNN})})$$

将 DNN 预测功率计算出的加权和速率 R_{DNN} 与以下基准进行比较:

- **WMMSE:** 在相同测试信道上运行 WMMSE 得到的和速率 (最优基准)
- **随机分配 (RAND):** $p_k \sim \mathcal{U}(0, P_{\max})$ (均匀随机功率)
- **满功率分配 (MP):** $p_k = P_{\max}$ (所有用户以最大功率发射)

- **Computation Time:**

- 测量 DNN 单次前向传播 (推理) 所需时间
- 与 WMMSE 算法的 Python 和 C 语言实现时间进行比较, 计算加速比 (Speedup)

4. **泛化能力测试 (Generalization Test):** 在用户数为 $K/2$ 的测试场景下评估模型性能, 而该模型是在用户数为 K 的数据上训练的 (用于考察模型对网络规模变化的适应能力)。

3.3 实验结果

3.3.1 Sum-Rate Performance

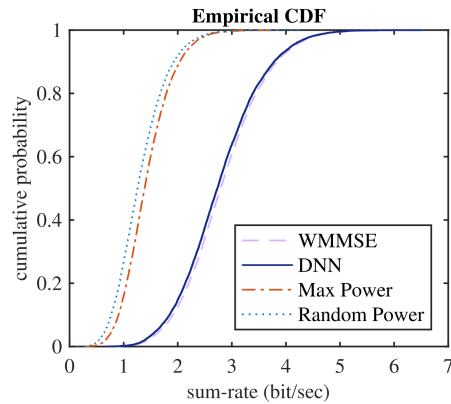


图 1: 高斯 IC 信道

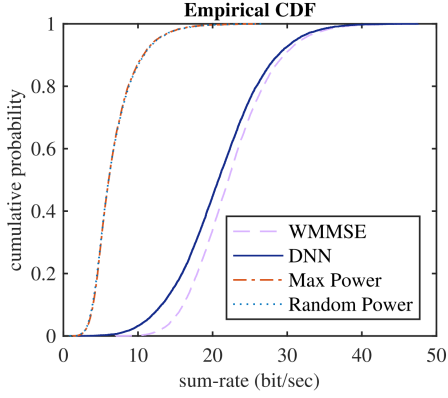


图 2: IMAC 信道

这两个图是 10000 个测试样本的 sum-rate 的累计概率分布图，基线是 p_k 全取最大值或随机值时的情况，可以看出 WMMSE 和 DNN 对 sum-rate 的优化结果基本一致。

3.3.2 Scalability Performance

表 1: Relative CPU Time and Sum-Rate

of users(K)	10	20	30
DNN sum-rate(wrt WMMSE)	0.972	0.9265	0.8566
DNN sum-rate(wrt Greedy)	0.9666	0.9187	0.8521
test time(wrt WMMSE,Python)	0.0032	0.0016	0.0012
test time(wrt Greedy)	0.0052	0.0011	0.0005
sum-rate($K/2$ user, wrt WMMSE)	0.9922	0.9278	0.8777

为了验证这种方法的可扩展性，又用 DNN 去学习了贪婪算法，以及 $K/2$ 的情况，表中是 DNN 输出的 sum-rate 相较于 WMMSE 和 Greedy 的值，基本上很接近了。而时间对比上 DNN 比 WMMSE 和 Greedy 快了将近 300 倍。

4 Future Work

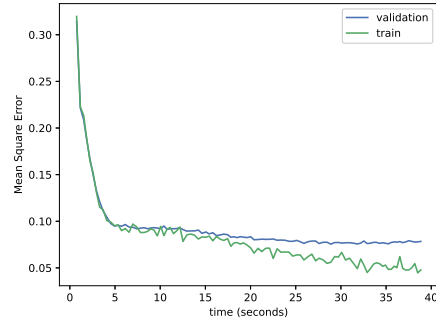
- DNN 是否可以处理测试分布与训练分布不一致的情况？
- 如何进一步降低 DNN 的计算复杂度？使用深度网络有什么理论依据吗？
- 如何将提出的方法推广到更具挑战性的问题，如 IC/I-MAC 的波束成形？

5 复现结果

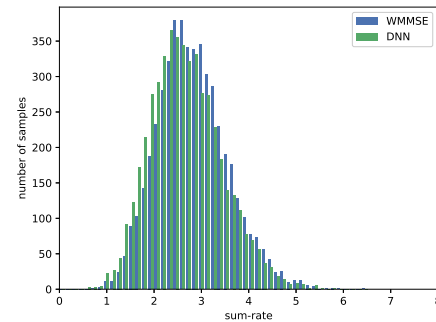
python3.6+tensorflow1.15.0+2080ti

5.1 Demo.py

图三中仅使用 25000 个样本对模型进行训练，MSE 的值随着模型训练时间增加而下降，最后收敛至 0.75 左右。模型最终可以达到 93.724% 左右的精确度（相较于 WMMSE 算法）。这里我跑出来的模型精度比 readme 文件里标注的 94% 还是要小一些，但感觉已经很接近了。即使在小样本和较短的训练时间的情况下，DNN 依然在准确率方面表现出较好的性能。

图 3: MSE 随模型训练时间变化，高斯 IC 信道， $K = 10$ ，25000 训练样本表 2: Relative CPU Time and Sum-Rate $K = 10$

算法种类	WMMSE	DNN	Max Power	Random Power
运行时间	8.368s	0.024s	/	/
Sum-rate	2.833	2.655	1.428	1.317
Ratio	100%	93.724%	50.414%	46.478%

图 4: 测试集 Sum-rate 分布，高斯 IC 信道， $K = 10$ ，训练样本 25000

表格二中，运行时间方面，DNN 要显著小于 WMMSE，算法计算效率大概提高了 350.6 倍，这说明采用 DNN 可以很大程度上提高功率优化分配的实时性。其次，Sum-rate，DNN 的优化结果相较于 WMMSE 算法可以达到 93.724%

的准确率，显著高于两条基线 Max Power（最大化功率分配）和 Random Power（随机功率分配）。图四是 $K = 10$ 的情况下高斯 IC 信道，2500 个测试集，用两种方法分别进行优化之后的 Sum-rate 分布。二者几乎相同，DNN 可能会在左尾出现轻微性能损失。

5.2 test.py

对原文中的表 1 进行复现，这时样本数据变成 100 万，其中的 99% 作为训练集，1% 作为测试集，不过这里没有从头训练模型，就是用的开源代码已经预训练好的模型。

表 3: Relative CPU Time and Sum-Rate $K = 10$

of users(K)	10	20	30
DNN Sum-rate(wrt WMMSE)	0.9819	0.9214	0.8605
test time(wrt WMMSE)	0.2222	0.0044	0.0024

运算时间方面比原文中的计算时间要满了很多，目前还没有找出原因。

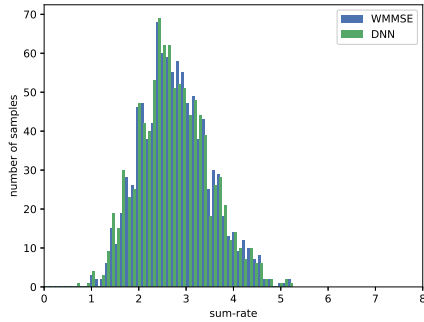


图 5: IC 信道 $K = 10$

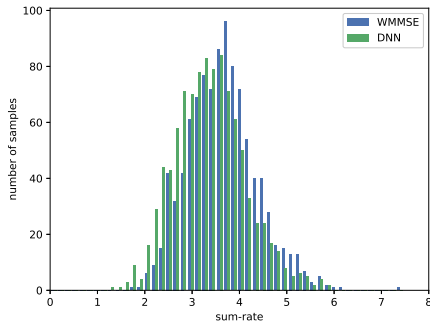


图 6: IC 信道 $K = 20$

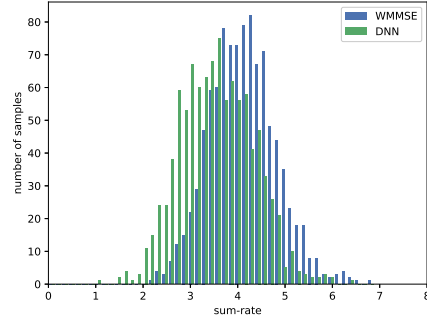


图 7: IC 信道 $K = 30$

文中提出的方法在预测准确性和计算效率方面展现出极佳的可扩展性。不过随着 K 的增大，准确率和实时性会下降。

5.3 论文复现——Sum-Rate Performance

本节对文献中提出的深度神经网络无线资源管理方法进行复现验证。重点对比分析了高斯干扰信道 (Gaussian IC) 和干扰多址信道 (IMAC) 两种场景下的性能表现。

5.3.1 高斯干扰信道 (Gaussian IC) 性能验证

图 8 展示了原文中高斯 IC 信道的累积分布函数 (CDF) 对比结果，其中包含了 WMMSE、DNN、最大功率分配 (Max Power) 和随机功率分配 (Random Power) 四种方法的性能对比。

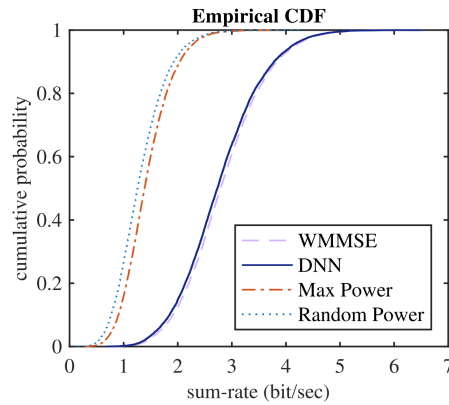


图 8: 高斯 IC 信道性能对比——原文结果 ($K = 10$)

图 9 和图 10 分别展示了我们的复现结果。从 CDF 曲线可以看出，DNN 方法的性能分布与 WMMSE 算法高度一致，远优于基线方法。

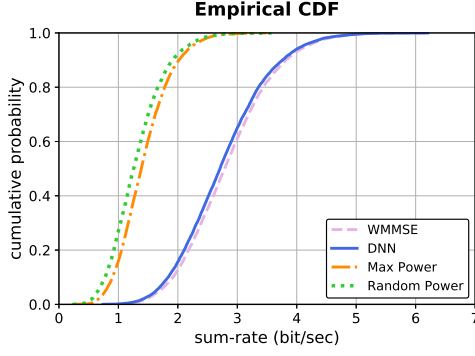


图 9: 高斯 IC 信道 CDF 对比——复现结果 ($K = 10$)

```
Step 5: Performance comparison and plotting
Calculating sum-rates for different schemes...
DNN achieves 97.42% of WMMSE sum-rate on average
CDF plot saved: /root/mycode/results/CDF_GaussianIC_K10.png
EPS format saved: /root/mycode/results/CDF_GaussianIC_K10.eps
Step 6: Plotting training curves
```

图 10: 高斯 IC 信道性能统计——复现结果 ($K = 10$)

表 4 给出了高斯 IC 信道的定量性能对比。结果显示，我们的复现实验中 DNN 相对于 WMMSE 的性能达到了 97.42%，与原文的 97.92% 仅相差 0.50%，验证了复现的高度准确性。

表 4: Gaussian IC Performance Comparison ($K = 10$)

Metric	Original	Reproduced	Diff.
DNN (w.r.t. WMMSE)	97.92%	97.42%	-0.50%

5.3.2 干扰多址信道 (IMAC) 性能验证

图 11 展示了原文中 IMAC 信道的性能对比结果，该场景考虑了多小区干扰环境下的更复杂资源分配问题。

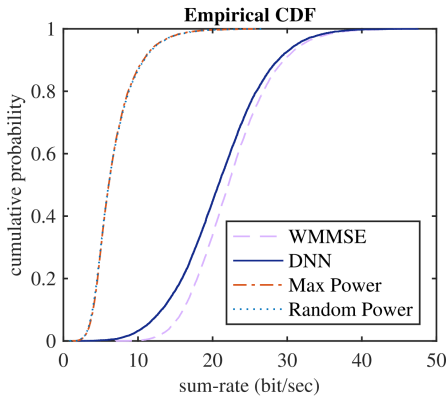


图 11: IMAC 信道性能对比——原文结果 ($N = 3, K = 24$)

图 12 展示了我们在 IMAC 场景下的复现结果。可以观

察到 DNN 方法在复杂的多小区干扰环境中仍能保持良好的性能。

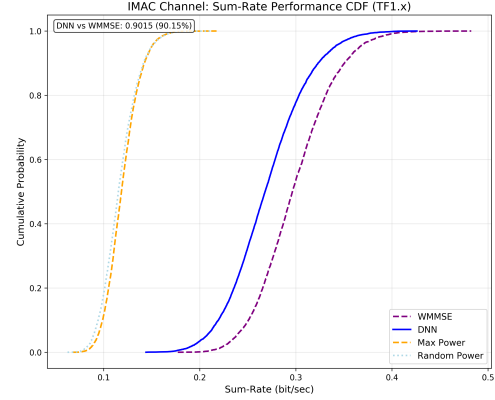


图 12: IMAC 信道性能对比——复现结果 ($N = 3, K = 24$)

表 5 给出了 IMAC 信道的性能对比结果。在更复杂的 IMAC 场景中，我们的复现结果显示 DNN 相对于 WMMSE 的性能为 90.15%，与原文的 93.02% 相差 2.87%。

表 5: Gaussian IC Performance Comparison ($K = 10$)

Metric	Original	Reproduced	Diff.
DNN (w.r.t. WMMSE)	93.02%	90.15%	-2.87%

5.3.3 复现结果分析

性能验证：从复现结果可以看出，本次实验成功验证了原文的主要结论：

- DNN 方法能够有效近似 WMMSE 算法的性能，在两种信道场景下都达到了 90% 以上的相对性能；
- 与基线方法（最大功率分配和随机功率分配）相比，DNN 方法展现出显著的性能优势；
- 复现结果与原文基本一致，验证了深度学习方法在无线资源管理中的有效性。

差异分析：观察到的性能差异主要源于以下因素：

- **随机性因素：**神经网络权重初始化、训练数据集划分等随机过程的不同；
- **实现细节：**超参数设置、优化算法收敛准则等实现细节的微小差异；
- **数值精度：**不同计算平台和深度学习框架的数值计算精度差异；
- **场景复杂度：**IMAC 场景相比 IC 场景更加复杂，对训练数据分布和网络参数更加敏感。

结论: 总体而言, 复现实验验证了原文方法的有效性和可重现性。IC 信道场景下 0.50% 的差异和 IMAC 信道场景下 2.87% 的差异均在合理的误差范围内。

5.3.4 代码开源

复现实验的完整代码存放在 GitHub 个人仓库中。代码仓库包含了数据生成、模型训练、性能评估等完整的实验流程, 以及详细的使用说明和参数配置。

代码仓库:

<https://github.com/zwxtime/Learn-to-Optimize-Wireless>

代码仓库的主要内容包括:

- 高斯 IC 和 IMAC 信道模型的数据生成脚本
- WMMSE 算法的 Python 实现
- 深度神经网络模型的 TensorFlow 实现
- 性能评估和可视化代码
- 实验配置文件和超参数设置
- 详细的 README 文档和使用教程